

Phase 3 Quick Reference

Quick reference guide for common Phase 3 operations

[Initialization](#)

[Session Management](#)

[Memory System](#)

[State Persistence](#)

[Template System](#)

[Integration Patterns](#)

[Common Tasks](#)

[Configuration](#)

Initialization

```
import (  
    "dev.helix.code/internal/session"  
    "dev.helix.code/internal/memory"  
    "dev.helix.code/internal/persistence"  
    "dev.helix.code/internal/template"  
)  
  
// Create managers  
sessionMgr := session.NewManager()  
memoryMgr := memory.NewManager()  
templateMgr := template.NewManager()  
  
// Set up persistence  
store := persistence.NewStore("./helixcode_data")  
store.SetSessionManager(sessionMgr)  
store.SetMemoryManager(memoryMgr)  
store.SetTemplateManager(templateMgr)  
store.EnableAutoSave(300) // 5 minutes  
  
// Load built-in templates  
templateMgr.RegisterBuiltinTemplates()  
  
// Restore previous state  
store.Load()
```

Session Management

Create and Start Session

```
sess := sessionMgr.Create("feature-name", session.ModeBuilding, "project-id")
sess.AddTag("api")
sessionMgr.Start(sess.ID)
```

Session Modes

```
session.ModePlanning      // Design and architecture
session.ModeBuilding      // Implementation
session.ModeTesting       // Quality assurance
session.ModeRefactoring   // Code improvement
session.ModeDebugging     // Issue investigation
session.ModeDeployment     // Release preparation
```

Session Lifecycle

```
sessionMgr.Start(id)      // idle → active
sessionMgr.Pause(id)      // active → paused
sessionMgr.Resume(id)     // paused → active
sessionMgr.Complete(id)   // active → completed
sessionMgr.Fail(id, reason) // active → failed
```

Query Sessions

```
all := sessionMgr.GetAll()
byProject := sessionMgr.GetByProject("project-id")
```

```
byMode := sessionMgr.GetByMode(session.ModeBuilding)
byStatus := sessionMgr.GetByStatus(session.StatusActive)
byTag := sessionMgr.GetByTag("api")
recent := sessionMgr.GetRecent(10)
```

Memory System

Create Conversation

```
conv := memoryMgr.CreateConversation("Feature Implementation")
conv.SessionID = sess.ID // Link to session
```

Add Messages

```
// User message
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage("Help me debug this"))

// Assistant message
memoryMgr.AddMessage(conv.ID, memory.NewAssistantMessage("Sure, let's look at it"))

// System message
memoryMgr.AddMessage(conv.ID, memory.NewSystemMessage("You are a debugg"))
```

Query Messages

```
all := conv.GetMessages()
userOnly := conv.GetByRole(memory.RoleUser)
recent := conv.GetRecent(10)
```

```
results := conv.Search("debug")
text := conv.ToText()
```

Conversation Limits

```
conv.SetMaxMessages(500)
conv.SetMaxTokens(50000)
total := conv.TotalTokens()
conv.Truncate(100) // Keep last 100 messages
```

State Persistence

Save Operations

```
store.Save()           // Save all
store.SaveSessions()    // Save sessions only
store.SaveConversations() // Save conversations only
store.SaveTemplates()   // Save templates only
```

Load Operations

```
store.Load()           // Load all
store.LoadSessions()    // Load sessions only
store.LoadConversations() // Load conversations only
store.LoadTemplates()   // Load templates only
```

Auto-Save

```
store.EnableAutoSave(300) // Every 5 minutes
store.DisableAutoSave()
enabled := store.IsAutoSaveEnabled()
interval := store.GetAutoSaveInterval()
```

Backup and Restore

```
// Backup
backupPath := "./backups/backup-" + time.Now().Format("20060102-150405")
store.Backup(backupPath)

// Restore
store.Restore(backupPath)
```

Storage Formats

```
store.SetFormat(persistence.FormatJSON) // Human-readable
store.SetFormat(persistence.FormatCompactJSON) // Smaller
store.SetFormat(persistence.FormatJSONGZIP) // Compressed
```

Template System

Create Template

```
tpl := template.NewTemplate("Handler", "HTTP handler", template.TypeCode)
tpl.SetContent(`func Handle{{name}}(w http.ResponseWriter, r *http.Request) {
```

```

        {{implementation}}
    }`)

tpl.AddVariable(template.Variable{
    Name:      "name",
    Required:  true,
    Type:      "string",
})

templateMgr.Register(tpl)

```

Template Types

```

template.TypeCode           // Code generation
template.TypePrompt        // AI prompts
template.TypeWorkflow       // Process templates
template.TypeDocumentation  // Documentation
template.TypeEmail          // Email templates
template.TypeCustom         // Custom use

```

Render Templates

```

// By name
result, _ := templateMgr.RenderByName("Function", map[string]interface{
    "function_name": "ProcessData",
    "parameters":    "data []byte",
    "return_type":   "error",
    "body":          "return nil",
})

```

Built-in Templates

```
templateMgr.RegisterBuiltinTemplates()

// 1. Function
// 2. Code Review
// 3. Bug Fix
// 4. Function Documentation
// 5. Status Update Email
```

Integration Patterns

Session + Conversation

```
// Create linked session and conversation
sess := sessionMgr.Create("feature", session.ModeBuilding, "project")
sessionMgr.Start(sess.ID)

conv := memoryMgr.CreateConversation(sess.Name)
conv.SessionID = sess.ID
```

Template + Conversation

```
// Generate prompt from template
prompt, _ := templateMgr.RenderByName("Code Review", vars)

// Add to conversation
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(prompt))
```

Complete Workflow

```
// 1. Create session
sess := sessionMgr.Create("auth", session.ModeBuilding, "api")
sessionMgr.Start(sess.ID)

// 2. Create conversation
conv := memoryMgr.CreateConversation("Auth Implementation")
conv.SessionID = sess.ID

// 3. Use template
code, _ := templateMgr.RenderByName("Function", vars)

// 4. Add to conversation
memoryMgr.AddMessage(conv.ID, memory.NewAssistantMessage(code))

// 5. Auto-saved by persistence
```

Common Tasks

Start New Feature

```
sess := sessionMgr.Create("feature-name", session.ModeBuilding, "project")
sess.AddTag("feature")
sessionMgr.Start(sess.ID)

conv := memoryMgr.CreateConversation("Feature: " + sess.Name)
conv.SessionID = sess.ID
```

Debug Issue

```
sess := sessionMgr.Create("bug-123", session.ModeDebugging, "project")
sess.SetMetadata("issue_id", "123")
sessionMgr.Start(sess.ID)

conv := memoryMgr.CreateConversation("Debug: Issue 123")
conv.SessionID = sess.ID

prompt, _ := templateMgr.RenderByName("Bug Fix", vars)
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(prompt))
```

Code Review

```
sess := sessionMgr.Create("review-pr-42", session.ModeBuilding, "project")
sess.AddTag("review")
sessionMgr.Start(sess.ID)

conv := memoryMgr.CreateConversation("PR #42 Review")
conv.SessionID = sess.ID

prompt, _ := templateMgr.RenderByName("Code Review", map[string]interface{}{
    "language": "Go",
    "code":     prCode,
})
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(prompt))
```

Configuration

Recommended Config (YAML)

```
persistence:
  storage_path: "./helixcode_data"
  format: "json-gzip" # Compressed
  auto_save: true
  auto_save_interval: 300 # 5 minutes

session:
  max_history: 100
  default_mode: "building"

memory:
  max_messages_per_conversation: 500
  max_total_tokens: 50000

templates:
  template_directory: "./templates"
  load_builtin: true
```

In Code

```
// Set limits
conv.SetMaxMessages(500)
conv.SetMaxTokens(50000)
memoryMgr.SetMaxMessages(500)

// Configure auto-save
store.EnableAutoSave(300)
```

```
// Set format
store.SetFormat(persistence.FormatJSONGZIP)
```

Performance Tips

```
// 1. Set limits
conv.SetMaxMessages(500)
conv.SetMaxTokens(50000)

// 2. Use GZIP for large data
store.SetFormat(persistence.FormatJSONGZIP)

// 3. Trim regularly
sessionMgr.TrimHistory(50)
memoryMgr.TrimConversations(50)

// 4. Auto-save interval (not too frequent)
store.EnableAutoSave(300) // 5 minutes

// 5. Delete old data
for _, sess := range sessionMgr.GetAll() {
    if sess.Status == session.StatusCompleted {
        sessionMgr.Export(sess.ID) // Backup first
        sessionMgr.Delete(sess.ID)
    }
}
```

[Full API Reference](#)

[Phase 3 Features](#)

[Example Projects](#)